On Some Implementation Issues for Value Prediction on Wide-Issue ILP Processors

Sang-Jeong Lee[†] and Pen-Chung Yew[‡]

[†]*Division of Information Technology Engineering, Soonchunhyang Univ., Chungnam, Korea*

[‡]*Dept. of Computer Science and Engineering, Univ. of Minnesota,Minneapolis, MN 55455*

[†]*sjlee@sch.ac.kr,* [‡]*yew@cs.umn.edu*

Abstract

In this paper, we look at two issues which could affect the performance of value prediction on wide-issue ILP processors. One is the large number of accesses to the value prediction tables needed in each machine cycle, and the other is the latency required to update stale values in the value prediction tables. We introduce a prediction value cache (PVC) which augments the instruction cache to hold the prediction values. Using the PVC, we not only can provide required bandwidth to access multiple prediction values needed in each machine cycle, but also allow us to decouple the value prediction from the critical path in the instruction fetch stage. We use a hybrid value predictor with dynamic classification to perform value prediction in the write back stage, and assume a realistic number of read/write ports, e.g. 2 read/write ports, with queues in their prediction tables. We found good performance for a 8-issue processor using simulations.

We also found that, in a 8-issue processor using SPECint95 benchmark programs, 36% of instructions will access the same value prediction table entry again within 5 cycles, and 22% of instructions will do that within 2 cycles. Unless the prediction tables can be quickly updated, especially for the Stride type and the Two-level type, those value predictions will get stale values and mostly resulted in mispredictions. We examine several schemes such as attaching an age counter and using speculative update to cope with the problem of delayed updates, but found them not as effective due to the latency required in dynamic classification. If such latency can be reduced, e.g. by using compiler analysis to determine access types at compiler time, the performance could be further improved.

1. Introduction

To achieve high performance on wide-issue superscalar processors, it is essential to exploit instruction level parallelism (ILP). But control dependences and data dependences are major hurdles to exploit ILP efficiently. Many speculative execution schemes have been proposed to deal with them. In general, branch prediction is used to handle control dependences and value prediction for data dependences. Value prediction is a technique that breaks true data dependences by predicting the outcome of an instruction, and executes speculatively its data-dependent instructions based on the prediction outcome. There are many studies about value prediction [2-5,7,9-11,15-18,21]. These studies show value prediction can provide accurate values for many programs, and good performance gain can be obtained.

As processors continue to increase the width of instruction fetch and issue rate for higher performance, some difficulties have emerged which could keep value prediction from being used effectively. One of them is the increased multiple accesses to the value prediction table needed in each machine cycle. The other is potential stale values in the prediction tables due to the delay needed to produce and verify updated values in the tables.

Most value prediction schemes assume there is unlimited bandwidth and allow as many simultaneous accesses to the prediction tables as needed in the instruction fetch stage. In wide-issue superscalar processors, e.g. 8- to 16-issue processors, we may need 8 to 16 simultaneous accesses to the value prediction table in each machine cycle. This will require very expensive hardware because it needs a large number of read/write ports to the value prediction table which may have an adverse effect on the clock cycle time. Most prior studies are thus focused only on the predictor algorithms and their potential performance gain. Some studies such as Gabbay et al. suggest using multiple interleaved banks with a fast distribution network to access the value prediction table [4]. However, potential bank conflicts may exist because several instructions could access the same bank simultaneously. Furthermore, it is quite difficult to implement multiple-bank predictors for context-based value prediction schemes, such as the two-level predictor [21] and the FCM (Finite Context Method) predictor [18].

0-7695-0622-4/00 $10.00 © 2000 IEEE

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

In a processor, the result of each instruction is produced after its execution. Therefore, the value prediction table can be updated for next prediction only after its execution. If the value prediction table is accessed in the instruction fetch stage and assume we have an execution pipeline with more than 4 stages, it will take more than four cycles to produce the updated values for the prediction table. In a 8-issue processors, there will have been more than 32 instructions executed during this period of time. It is very likely that the same instruction will need to fetch the same value prediction table entry again before it has been updated (see section 3.2 for some measured data). This mostly will cause incorrect value predictions.

In this paper, we address those implementation issues for wide-issue architectures. We augment the instruction cache with a prediction value cache (PVC) to hold prediction values for each instruction in a cache block. It not only allows multiple accesses to the prediction values in each machine cycle, but also allows us to move the value prediction from the instruction fetch stage, and move it to a later pipeline stage after the update values are produced. We also propose a hybrid prediction scheme with dynamic classification which is similar to the scheme of Rychlik et al. [15]. Value prediction is carried out in the writeback stage where the updated values are produced. We assume each prediction table has only 2 read/write ports with queues. In addition, we study several schemes to reduce the potential stale value accesses using execution-driven simulations and SPECint95 benchmarks.

The rest of the paper is organized as follows: Section 2 describes related work on the value prediction schemes. In section 3 we present the problems of value prediction in wide-issue processors. Section 4 presents our decoupled value prediction scheme with dynamic classification. Section 5 summarizes our evaluation methodology, and in section 6 we present our simulation results and analysis. Finally, section 7 offers some conclusions.

2. Related work

For value prediction, several schemes have been proposed. They include last value prediction [10], stride prediction [3], context-based prediction [18,21], and hybrid value prediction [15,18,21]. Last value prediction predicts the result value of an instruction based on its most recently generated value. The predictor stores the last value produced by the instruction, and uses the value to predict the next result if its confidence counter is greater than a threshold. The confidence counter is incremented if the prediction is correct and decremented if not. A stride predictor predicts the value by adding the most recent value to the difference of the two most recently produced values. This difference is called stride. Last value prediction can be considered a special case of stride value prediction with zero stride. Context-based predictors predict the value based on the repeated pattern of the last several values observed. FCM [18] and two-level [21] predictors belong to this category. The two-level value predictor consists of two tables: a value history table (VHT) and a pattern history table (PHT). The VHT stores up to four distinct values generated by the instruction and keeps track of the order in which they are generated. The value history pattern records the pattern of the last few outcomes of the instruction. Each pattern is composed of a series of binary coded indices into the four data value fields. This value history pattern is used to index the PHT that has four up-down saturating counters corresponding to the data value fields. The counter is incremented or decremented according to the prediction result. Prediction is made only if the maximum counter value is greater than or equal to a threshold value. Its operation looks like a two-level branch prediction. Each of the predictors mentioned above shows good performance for certain data value sequences, but bad for others. Therefore, some hybrid predictors are proposed by combining several predictors. Wang et al. proposed a hybrid predictor that combines a stride predictor and a two-level value predictor [21]. Rychlik et al. [15,16] combine a last value predictor, a stride predictor and a FCM predictor. The choice of a predictor for each instruction is guided by a dynamic classification mechanism. They noticed that updating the value prediction tables after the real values are produced, instead of an ideal immediate update after each prediction table access, can result in lower prediction rate, hence, lower performance. This is because the subsequent instructions may use the stale prediction values before they are updated. They also developed a selective reissue core (SRC) to recover after mispredictions [16]. Calder et al. examine selective value prediction schemes based on the confidence counter value and the instruction types [2].

In spite of those studies on value prediction, there are few studies focusing on the bandwidth required on prediction tables for multiple value predictions, and on the stale prediction value due to the delayed update. Gabbay et al. have shown the limitation of multiple accesses to the value prediction table using a realistic implementation [4]. They proposed a highly-interleaved prediction table with a fast distribution network to support high instruction issue rate. Recently, we examined a decoupled value prediction scheme on trace processors [9]. We use a trace cache to hold prediction values and to decouple value prediction from the instruction fetch stage.

Heil et.al proposed a branch predictor by correlating data values to improve the accuracy of conditional branch prediction [6]. In their paper, they keep a dynamic count

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

of the occurrences of the fetched branch to detect outstanding instances of each branch. Skadron et al. discussed speculative update in branch prediction and show the effect of prediction accuracy and update time on overall performance [19]. Rakvic et al. proposed a completion time multiple branch predictor to reduce the complexity and latency of the instruction fetch stage [12].

3. Motivation

To show the problems of limited multiple accesses and stale prediction values on the prediction table, we simulate several schemes based on the hybrid predictors developed by Wang et. al [21].

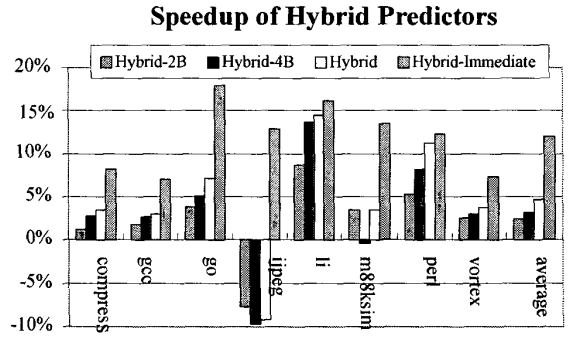


Figure 1. Speedup over a baseline architecture for several hybrid value prediction schemes

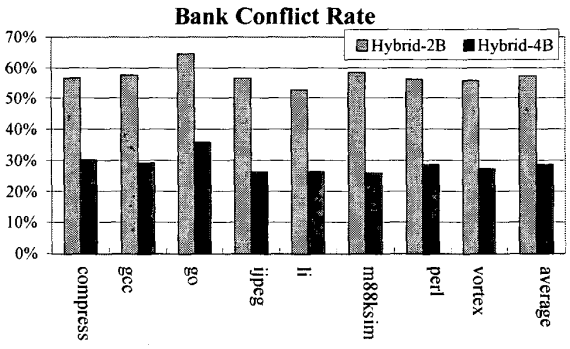


Figure 2. Bank conflict rates for the hybrid predictor with two-bank and four bank prediction table

Figure 1 shows the speedup over a baseline architecture without value prediction for several hybrid predictor schemes in a 8-issue processor (the simulation methodology is described in section 5.). In Figure 1, Hybrid-2B and Hybrid-4B use hybrid predictors with two-bank and four-bank prediction tables, respectively, to allow simultaneous accesses in each machine cycle. Hybrid uses a hybrid predictor which allows unlimited simultaneous accesses to the prediction table in each machine cycle. Hybrid-Immediate uses a predictor which updates its prediction table instantaneously with unlimited simultaneous accesses to the prediction table in one machine cycle.

3.1 The problem of multiple value access

From Figure 1, we can see that the banked predictors will degrade the performance comparing to Hybrid, mostly, due to bank conflicts. When bank conflicts occur, the prediction table cannot be accessed or updated. This leads to intermittent value predictions. Also, in the case of bank conflicts during updates, the prediction table may keep the stale values longer reducing its prediction accuracy. In the case of ijpeg, the scheme with value prediction shows lower performance because of the negative impact from misprediction penalties.

Figure 2 shows the bank conflicts rates. We interleave the prediction values among the banks based on the addresses of the instructions which will produce these values. In Figure 2, we can see that banked prediction tables may suffer high conflict rates. It makes them difficult to attain high overall performance.

3.2 The problem of stale prediction value fetch

Even if we ignore the effect of restricted simultaneous table accesses, there is still a great performance gap between Hybrid and Hybrid-Immediate in Figure 1. The main reason for this gap is due to the stale values in the value prediction tables. As the processor increases instruction fetch and issue width for higher ILP, the number of machine cycles between two successive value prediction table lookups to the same table entry could be substantially reduced. Furthermore, the increased number of pipeline stages required by a higher clock rate will increase the number of cycles between the actual prediction value update, which usually occurs in a later stage of the pipeline, and the prediction value usage, which usually occurs in an early stage of the pipeline. Therefore, many instructions may access stale prediction values from the table before they can be actually updated. This effect can cause serious performance degradation because stale values mostly are incorrect, especially for stride and context-based predictors which need up-to-date values for correct predictions. Figure 3 shows the distribution of the number of clock cycles between two consecutive table lookups to the same table entry on a 8-issue processor. The left graph shows the distribution cycles for all SPECint95 benchmarks. The right one

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

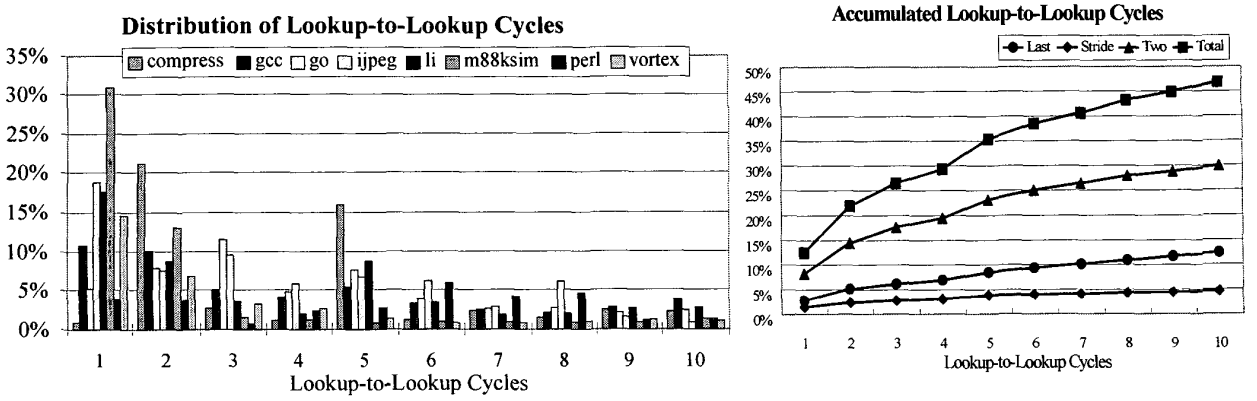


Figure 3. Distribution of clock cycles between two consecutive table lookups to the same table entry

shows the accumulated distribution cycles of them for each prediction types which are guided by dynamic classification mechanism [9,15]. On average, 36% of instructions access the same table entry again within 5 cycles (Among them, the last, the stride and the two level types are 9%,4%,23%, respectively). 22% of the instructions access the same table entry within 2cycles (Among them, the last, the stride and the two level types are 5%,2%,15%, respectively). These instructions will have a high misprediction rate if values in the table are stale.

In this paper, we propose several techniques to mitigate these problems. For the required multiple accesses to the prediction table in each machine cycle, we propose using a prediction value cache (PVC) in conjunction with the instruction cache. The organization of PVC is exactly the same as the instruction cache. Hence, the prediction value of each instruction can be fetched exactly the same way the instructions are fetched from the instruction cache.

To mitigate the problem of stale prediction values, we associate an age counter with each prediction value in PVC to identify potential stale prediction values. The age counter gives an indication of how many times the prediction value has been accessed before the prediction value is updated.

We also use a value prediction scheme which is decoupled from the instruction fetch stage [9]. This allows value prediction to be done in a later stage and shorten the time between the value prediction updates and their usages.

4. Decoupled value prediction scheme

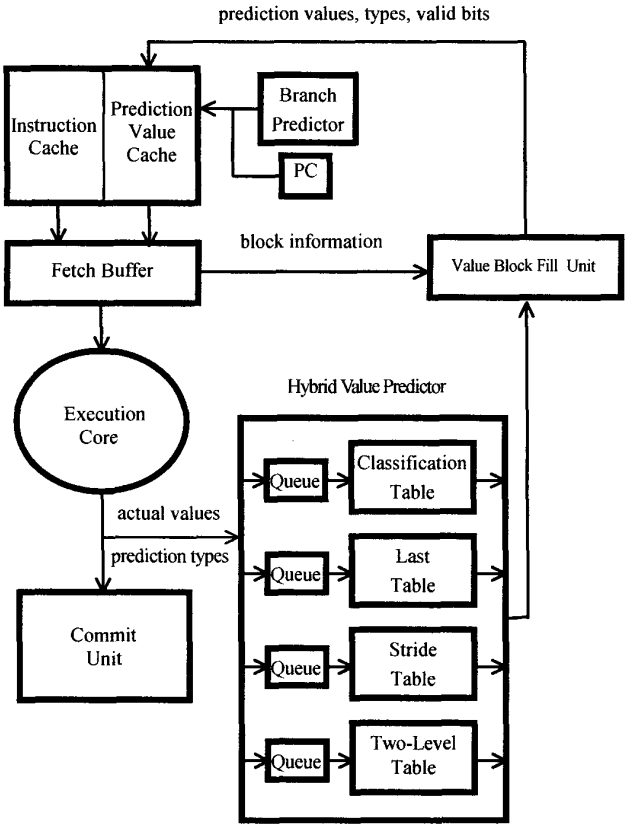
In this section, we describe the proposed decoupled value prediction scheme and a value predictor using a dynamic classification mechanism. Also we present the schemes using the age counter and speculative updates.

4.1. Decoupled value prediction scheme

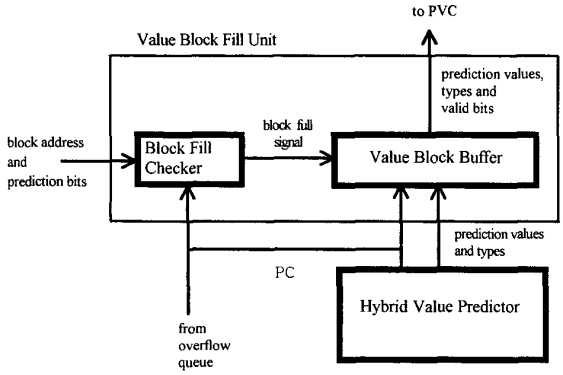
Figure 4 shows the main components of our decoupled value prediction scheme. It has a prediction value cache (PVC) associated with the instruction cache. PVC is organized in the same way as the instruction cache in terms of its block sizes, set associativity and address mapping scheme. Each instruction cache block is augmented with one PVC block, and each instruction in the instruction cache block has a corresponding entry in the PVC block. Each PVC entry contains the prediction value for the result of the corresponding instruction, its prediction type and a valid bit. Augmenting the instruction cache with PVC provides a very simple mechanism to access multiple prediction values without the complexity of having to access both the branch prediction tables and the value prediction tables in the instruction fetch stage as in other proposed schemes. It also avoids potential bank conflicts among the multiple accesses to the value prediction tables since both the instruction cache and PVC have the exact same organization.

As opposed to the previous schemes, our value predictor is decoupled from the instruction fetch stage and is accessed when the instructions complete their execution (details in section 4.2). The value predictor used is a hybrid predictor which includes: a last value predictor, a stride predictor, a two-level predictor and a classification table for dynamic classification. We assume that each predictor table has only two read ports and two write ports. Since there could be up to 8 instructions accessing and updating the prediction tables for a 8-issue processor, we provide a queue in front of each prediction table (see Figure 4 (a)). Because the predicted values are only advisory and the chance for a queue being full is small

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Figure 4. (a) Decoupled value prediction scheme (b) Value block fill unit

if the queue size is chosen properly, to simplify the hardware design, we assume further accesses to a prediction table are dropped when its queue becomes full. The prediction values in PVC may become stale when this happens. However, our simulation shows the overflow rate of the queue is negligible for most benchmarks (details in section 6.3). According to the prediction type, which is determined by the classification table and is stored in each PVC entry, a request is sent to update the prediction table during the writeback stage with the result its instruction just produced. It also produces a prediction value which is sent to the corresponding PVC entry through VFU (value block fill unit) after the prediction table lookup (see Figure 4 (b)). VFU receives the block information after instructions are decoded. It allocates one block for each corresponding instruction cache block. The information VFU received includes the block address, which is the starting address of the cache block, and a prediction bit for each instruction in the block. The prediction bit is set when the instruction requires a predicted value, i.e. requires value prediction. When VFU receives prediction values, prediction types and the addresses of the instructions from the hybrid value predictor, VFU assembles these information to form a block. When all of the information becomes available in a block, using the block fill checker, VFU forwards the block to the PVC. When the queues overflow, to simplify our scheme, we assume that the requests to the hybrid prediction table are dropped and VFU is notified. The corresponding instructions can thus miss the updated prediction values. However, in our simulations, overflow in queues rarely happen when we use a queue size of 32.

4.2. Instruction pipeline design

There are many ways to implement an instruction pipeline which incorporate the value prediction scheme proposed above. It is beyond the scope of this paper to come up with an optimal design. However, to facilitate our simulation study on the performance of the proposed scheme, we assume an instruction pipeline similar to the one used in the SimpleScalar simulator [1] described as follows.

In the instruction fetch stage, the predicted result values from the PVC are read at the same time when the instructions are fetched from the instruction cache. These predicted result values are forwarded to the data-dependent instructions waiting in the reservation stations through the fetch buffer (see Figure 4) after the instructions are decoded in the dispatch stage. In the dispatch stage, the block information which consists of the block address and valid bits are sent to the VFU. When all the operands of an instruction are available in the reservation station (regardless of whether their values are predicted or not) and there is an available functional unit, the instruction is scheduled and executed. As in most value prediction schemes which require such speculative execution, the reordering buffer is used to keep both speculative and non-speculative results. After the actual

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

results are produced, the value prediction is verified in the writeback stage. If the prediction is correct, the execution is continued without any interruption. If the prediction is incorrect, the execution of the data-dependent instructions which have already been issued will be invalidated and reissued with the correct values. We use the selective reissue model proposed by Rychlik et al. [16]. The actual results and the correctness of the prediction are inserted into the queues according to their prediction types. They will then be used to update the value prediction tables and to obtain the next prediction values.

When a predictor in the hybrid value predictor receives a request from the queue, it will process the request according to its prediction type. The predictor updates the state and the confidence counter of the corresponding entry in the prediction table according to the produced result. If the previous prediction is incorrect, it may need to store the new value in the table according to the prediction type. As soon as it updates the prediction table, it will predict the next value according to the prediction type and send the result to VFU. VFU collects prediction result of a instruction and assemble a block. After a block is assembled, it is forwarded to PVC.

4.3. A decoupled hybrid value predictor using dynamic classification

Our predictor is similar to the one used in [9]. It partitions the requests into several prediction types. The classification is done dynamically with the help of the classification table. The table contains five fields: tag, state, prediction value, stride and class fields. The class field indicates one of the four types: Last, Stride, Two-level, and Unknown. The state field of the classification table is initially set to the initial state. When the value predictor encounters an instruction for the first time, it forwards the instruction to the classification table (see Figure 4). The initial state changes to the transient state. The result value is stored in the value field and the class is set to the Unknown type. If an entry in the classification table is in the transient state, a stride value is calculated by subtracting the old value in the value field from the new result value of the instruction. The new result value and the stride are stored in the value field and the stride field in the classification table, respectively. The state is then changed to the classified state. If an entry is in the classified state, its stride value will be calculated again. If the newly calculated stride value is the same as the stride value stored in the stride field and the stride value is zero, the prediction type of the instruction is set to the Last type. If the stride values are the same but the stride value is not zero, the type of the table is then set to the Stride type. The prediction type and the predicted value are sent to PVC. Next time, the instruction is predicted by using the corresponding predictor type. If the newly calculated stride value is not the same as the stride value stored in the stride field, the instruction is classified as the Two-level type.

Once the prediction type of an instruction is determined, the instruction is removed from the classification table, and its information is forwarded to the corresponding prediction table. Its future predicted values are obtained by accessing the corresponding prediction table specified by its prediction type. A confidence counter is associated with each entry in the prediction tables. If the value of a confidence counter becomes zero after repeated mispredictions, the predictor can no longer accurately predict values for the instruction. In this case, the instruction is removed from the predictor table, and the type of prediction is reset to the Unknown type. This instruction needs to be reclassified using the classification table.

Our predictor has several differences from the one used in Rychlik et al. [15]. First, for the classified type, the decoupled scheme does not need another table because the type is stored in PVC. Second, to classify an instruction, Rychlik et al. stores three distinct values in the classification table, but our scheme stores only one value and one stride. Finally, our scheme always tries to predict the value, but in their scheme, when the instruction is removed from the FCM predictor, the instruction is no longer predicted. Our decoupled predictor provides not only the hybrid prediction mechanism but also multiple prediction tables for simultaneous multiple value predictions.

4.4. Age counters and speculative update

As mentioned in section 3.2, stale prediction values can cause misprediction and can have a significant impact on performance. Therefore, we investigate schemes to reduce this effect in our decoupled value prediction scheme, especially for the Stride type and the Two-level type. We introduce an age counter in PVC to determine the "staleness" of a predicted value. Initially the age counter is set to zero. Whenever a prediction value is fetched in PVC, its age counter is incremented. When the predicted value in PVC is updated for the predictor, the counter is decremented. If the value of a counter is zero, it means that the value in PVC isn't stale. If the value of the age counter is not zero, some action is needed to cope with stale prediction value.

In this paper, we examine three schemes to deal with accesses to potential stale prediction values. In the first scheme, for the stride or the two-level type, if its age counter value is not zero, we don't use the prediction value in PVC because its stale prediction value may cause misprediction, i.e. we don't perform value prediction in such cases to avoid misprediction penalty. But this

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

scheme will bypass many opportunities for value prediction because the stride type and the two-level type are the dominant types in most of our benchmarks. In the second scheme, we exclude only the stride type for speculative execution because it is most likely to cause mispredection if it has stale values. In the third scheme, we use a speculative update scheme for the stride type. In this scheme, instead of skipping value prediction, we update the prediction values speculatively using the value of the age counter. The age counter value actually shows the number of outstanding accesses to the prediction value. Hence, we can obtain correct prediction values by adding the value in PVC with an adjusted value calculated by multiplying the stride value with the value of the age counter. The obtained prediction value is updated in PVC, and the age counter is reset to zero. We also mark that instruction as a speculative update candidate. When this kind of instruction is encountered again, the prediction value is fetched for speculative execution and then the new prediction value is calculated by adding the value with the stride, and update this new value in PVC. In section 6.1 , we will discuss the performance of each scheme.

5. Evaluation methodology

In this section, we describe our performance evaluation methodology. First, we explain the machine model used in our simulation study. Then, we describe the simulation set up and the benchmark programs used.

5.1. Machine model

We use a 8-issue baseline architecture. Table 1 summarizes some of the parameters used in our baseline architecture. The model has a high instruction fetch bandwidth and a large instruction window as well as large on-chip and off-chip caches. The hybrid branch predictor uses a gshare branch predictor with 16K entries and 8-bit history, and a 16K-entry bimodal branch predictor. The parameters used in PVC and the proposed hybrid value predictor are listed in Table 2. The PVC has 64KB and direct-map cache with 32 bytes per block. The hybrid value predictor has a 8K-entry value history table (VHT) and a 8K-entry pattern history table (PHT). The value predictor in our proposed scheme has a 4K-entry Last table, a 1K-entry Stride table, a 8K-entry Two-level table and a 1K-entry Classification table. These table sizes are derived from the dynamic classification rates of the predicted instructions (see the measured rates in section 6.3). The overall table size of our predictor is roughly 7% larger than the size of a 8K-entry hybrid predictor.

5.2. Evaluation method

The instruction pipeline of our machine model is based on SimpleScalar with fetch, dispatch, issue, execution, writeback and commit stages [1]. The model supports out-of-order issue and execution based on Register Update Unit (RUU). RUU is the unit that combines the instruction window for instruction issue and the reorder buffer for instruction commit [8]. The value predictor updates the prediction tables with the computed value after the writeback stage. The value misprediction penalty of the model is one clock cycle. When a value misprediction is detected in the writeback stage, only its data-dependent instructions which have already been issued are reissued. We use selective issue core model [16], in which the speculative result is not broadcast on the result bus.

Simplescalar/Alpha 3.0 tool set is used [1]. We add the value predictors and our decoupled scheme on the Simplescalar's sim-outorder simulator. The SPECint95 used in our simulation is listed in Table 3. The table shows the input, the number of instructions executed, the percentage of register-write instructions that need value prediction, the IPC of the baseline architecture for each program.

Table 1. Machine configuration for a baseline architecture

	Parameter	Value	Comments
Processor Core	RUU size	256	Instruction window
	LSQ size	64	Load-store queue
	Fetch width	8 instruction/cycle	
	Decode width	8 instruction/cycle	
	Issue width	8 instruction/cycle	
	Commit width	8 instruction/cycle	
	Functional units	8 int alu(1), 4 fp add(2), 2 int mult(3) div(12), 2 fp mult(4) div(12),	( ) is latency
Branch Predictor	BTB	2K, 2-way	Branch target buffer
	Hybrid Branch Predictor	gshare 16K, 8bit history, bimodal 16K	Gshare + Bimodal
	RAS	32	Return addr. stack
Cache	L1 D- cache	512K, 2-way, 32 byte blocks, 4 ports, 3-cycles latency	Data cache
	L1 I- cache	512K, direct-map, 32 byte blocks	Instruction cache
	L2 cache	unified, 1M, 4-way, 64 blocks	Secondary cache

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

Table 2. Prediction value cache and value predictor configuration

	Configuration
Prediction Value Cache	64K, direct-map, 32 byte block
Hybrid Value Predictor	8K VHT(Value History Table) 8K PHT(Pattern History Table) 4 history data values
Decoupled Hybrid Value Predictor	4K last table, 1K stride table, 8K two-level table, 1K classification table, 32 queue size

Table 3. SPECint95 benchmark set

Program	Input	# of instr. executed	% of reg. write instr.	IPC of base
compress	10000 e 2231	35,726,960	71%	2.56
gcc	jump.i	40,268,745	71%	1.97
go	5 9	81,818,896	83%	1.69
ijpeg	tinyrose.ppm	74,807,331	75%	3.04
li	queen6.lsp	41,712,945	66%	1.87
m88ksim	dhry.big.100	101,391,205	73%	2.87
perl	scrabbl.in	40,465,023	67%	2.38
vortex	persons.250	65,086,221	61%	3.38
Average		60,159,666	71%	2.47

6. Performance analysis

In this section, we examine the performance of our proposed decoupled value prediction scheme. We compare our scheme with a hybrid prediction scheme similar to [21]. We also study the effectiveness of the age counter and speculative update. We examine the speedup over the baseline architecture and then look at other parameters such as predictability, the distribution of the prediction types, and, finally, the performance impact of the size of PVC.

6.1. Speedup

Figure 5 shows the speedup over the baseline architecture. Hybrid-2B and Hybrid-4B use the hybrid predictor schemes similar to [21] with a two-bank and a four-bank prediction table, respectively, to allow multiple accesses in one machine cycle. Hybrid is the hybrid predictor with no limitation on the number of accesses to the prediction table in each machine cycle. Hybrid-Immediate is the predictor that updates prediction table instantaneously with no limitation on the number of accesses to the value prediction table in each cycle. PVC uses our decoupled scheme with only 2 read/write ports in the prediction tables. In Figure 5, the average speedups are 2.4%, 3.1% for Hybrid-2B and Hybrid-4B, 4.6%, 11.9% for Hybrid and Hybrid-Immediate, and 6.3% for PVC. PVC shows a better performance than Hybrid, which has no limitation on the number of accesses to the prediction tables. It is because PVC with a dynamic classification capability can predict values more accurately than Hybrid. But PVC cannot reach the performance level of Hybrid-Immediate because of the latency required to classify the prediction types and to update the prediction values. The performance of ijpeg using hybrid schemes shows negative impact of misprediction (ijpeg has the highest miss prediction rate among SPECint benchmarks).

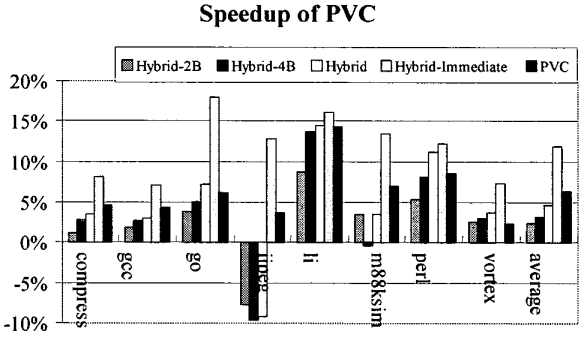


Figure 5. The speedup over the baseline architecture

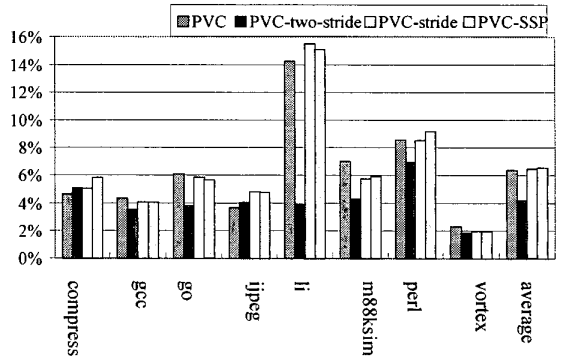


Figure 6. The speedup of PVC predictors over the baseline architecture

Figure 6 shows the speedup over the baseline architecture for several PVC schemes. PVC-two-stride is the scheme which does not perform value prediction for those instructions whose value prediction type is either stride or two-level and their age counter values are not zero. PVC-stride is the scheme which does not perform

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

value prediction for instructions whose value prediction type is stride and their age counter values are not zero. PVC-SSP is the scheme which updates speculatively for the stride type when their age counter values are not zero.

In Figure 6, on average, the performance of PVC-stride and PVC-SSP is slightly better than PVC. But PVC-two-stride is worse than PVC because the performance gain from using PVC-two-stride for li is much worse than that from using PVC. It is because PVC-two-stride skips too many value predictions. In the case of compress and ijpeg, PVC-stride and PVC-SSP show somewhat better performance, but m88ksim does not perform well. The main reason that the age counter and speculative update did not show the expected performance gain is that the number of loop iterations in most benchmarks is too small to cover the latency required to determine the value prediction types dynamically, and hence, to exploit the benefit of the age counter and speculative update. We believe that if compiler analysis can be used to determine the value prediction types, the age counter and the speculative update will be much more effective. However, that is beyond the scope of our paper.

6.2. Prediction coverage, predictability and prediction accuracy

We can use several different parameters to examine the performance of a value predictor. *Prediction coverage* is defined as the percentage of instructions which attempt value predictions among all instructions with register writes. (i.e. they exclude those value predictions which encounter below threshold confidence counter values or PVC misses) *Predictability* is the percentage of correct predictions among all instructions with register writes. *Prediction accuracy* is the percentage of correct predictions within prediction coverage. (among all those instructions which actually obtain prediction values)

Figure 7 shows the prediction coverage, predictability and the misprediction rate (*Misprediction rate* is the percentage of incorrect predictions among all instructions with register writes) for the hybrid predictor and the proposed decoupled predictor. The lower part of each bar in Figure 7 indicates the predictability and the upper part of each bar shows the misprediction rate. Therefore the sum of both is the prediction coverage. Figure 8 shows the prediction accuracy. PVC-Immediate uses the PVC scheme but allows immediate updates. From Figure 7 and 8, the average prediction coverage of Hybrid (73.4%) is higher than the one of PVC (45%). The reason for this is that PVC needs time to classify the prediction type. During that period of time it cannot predict value. However, because of dynamic classification, the prediction accuracy of PVC (76.3%) is higher than that of Hybrid (65.4%). The prediction coverage of Hybrid-

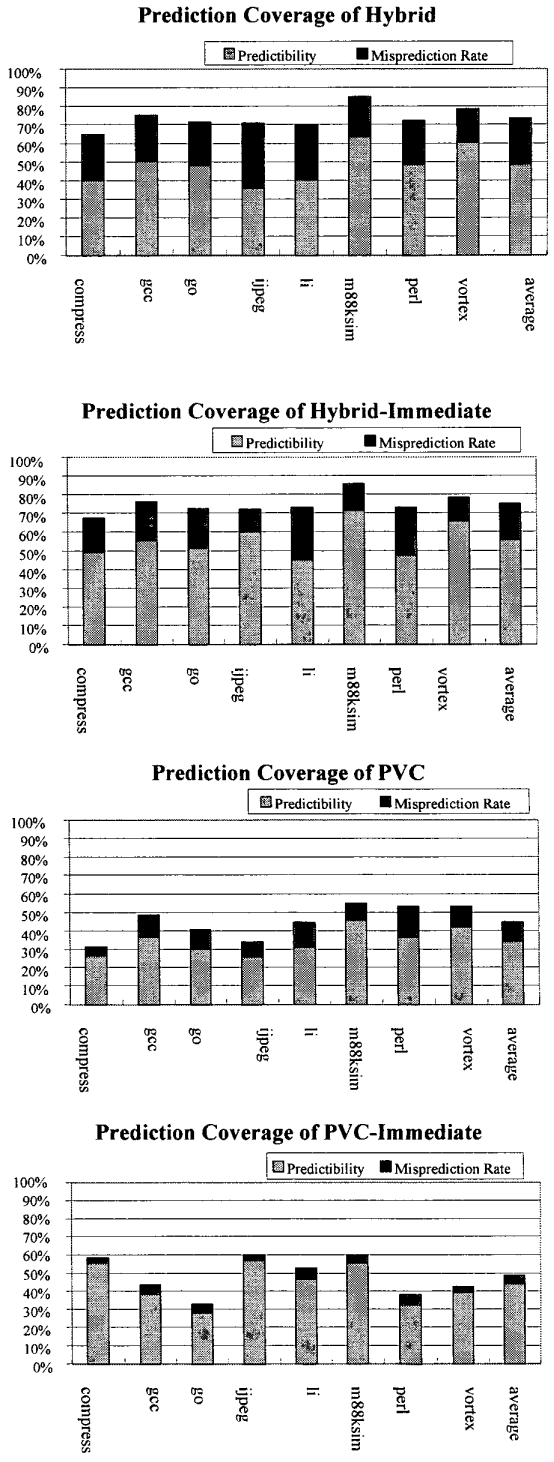


Figure 7. The prediction coverage and predictability of Hybrid and PVC schemes.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

Immediate (74.6%) similar to that of Hybrid (73.4%), but the prediction accuracy of Hybrid-Immediate (74.3%) is higher than that of Hybrid (73.4%). Also, the prediction coverage of PVC-Immediate (48.4%) is similar to that of PVC (45%), but the prediction accuracy of PVC-Immediate (90.4%) is much higher than that of PVC (76.3%). This difference of prediction accuracy is mostly due to the problem of stale prediction values.

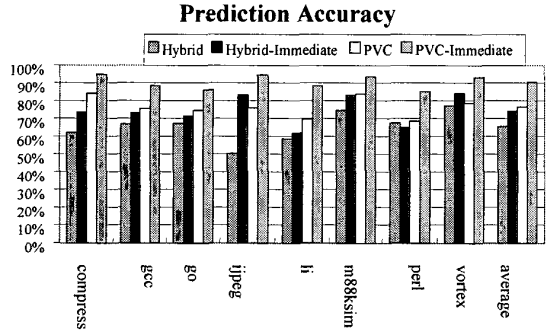


Figure 8. The prediction accuracy

6.3. Distribution of prediction types

Figure 9 shows the distribution of prediction types for PVC and PVC-Immediate. For PVC, 37.0% of the instructions which perform value prediction have the Last type, 8% have the Stride type, 43.5% have the Two-level type, 8.5% have the Unknown type, and 2.5% of the instructions cannot be predicted due to queue overflow in the predictors. For PVC-Immediate, on average 25.4% of instructions which perform value prediction have the Last type, 13.4% have the Stride type, 32.2% have the Two-level type, 28.8% have the Unknown type. The percentages of the Last and the Two-level for the PVC are higher than those of PVC-Immediate, but the percentages of the Stride and the Unknown for PVC are lower than those of PVC-Immediate. Because PVC may contain stale values, the instructions of the Stride type could mispredict the value repeatedly. Hence, Stride type could be classified as the Two-level type after several mispredictions. Also, in PVC scheme, the conversion from one type to another takes more time than in PVC-Immediate scheme. This tendency increases the percentages of the Last and Two-level types, and decreases the percentage of the Unknown type in PVC scheme comparing with the PVC-Immediate scheme. Except for ijpeg, the overflow rate for each queue is below 2%. But the overflow rate for ijpeg could be as high as 16.1%.

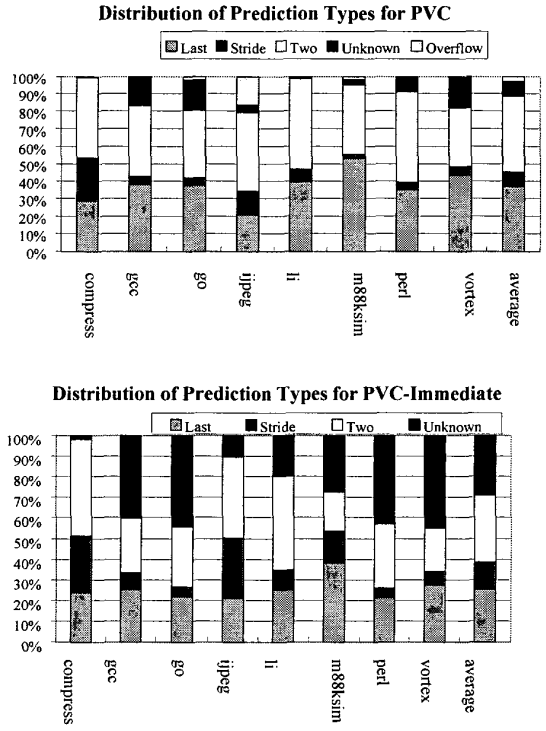


Figure 9. Distribution of Prediction Types for PVC and PVC-Immediate

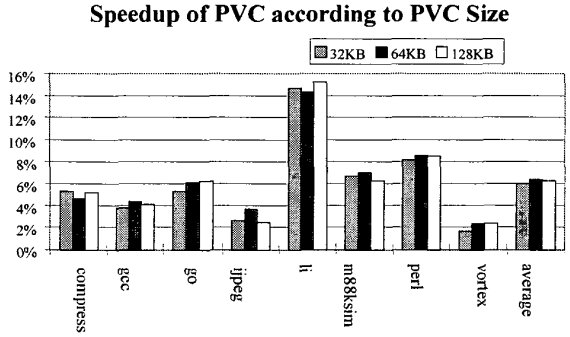


Figure 10. Speedup of PVC according to PVC size

6.4 Performance impact of PVC size

Figure 10 shows the speedup of the PVC scheme with the size of PVC being 32KB, 64KB and 128KB. Average speedups are 6.0%, 6.4%, and 6.3%. Figure 11 shows the PVC miss rate for each PVC size. On average, the miss rate is 5.4% for 32KB, 3.0% for 64KB and 0.8% for

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

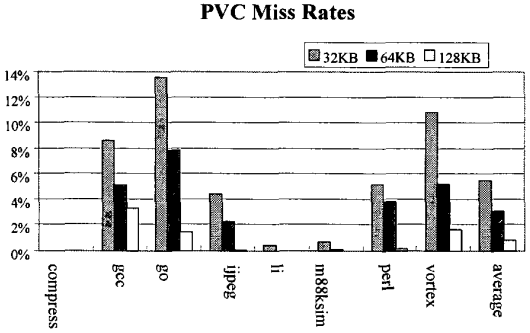


Figure 11. PVC miss rates with PVC size

128KB. Even though the miss rate of PVC decreases as the size of PVC increases, the performance is not very sensitive to the PVC size. This is because not all correct value predictions can improve the bottom-line performance. Only those correct value predictions which actually break the data dependences for those dependent instructions issued in the same cycle or within the time window they can actually improve overall performance.

7. Conclusions

In this paper, we propose a value prediction scheme with dynamic classification using a prediction value cache (PVC) for wide-issue processors. By augmenting instruction cache with a PVC and decouple value prediction from the instruction fetch stage, as well as providing multiple value prediction tables and queues, we can provide effective value prediction using realistic 2 read/write ports in prediction tables for a wide-issue (e.g. 8-issue) processor. We obtain some performance results using Simplescalar Alpha 3.0 tool set on the SPECint95 benchmarks.

Our value prediction scheme provides an average speedup of 6.3% over a baseline architecture without value prediction. This performance is better than the previous hybrid schemes which allow unlimited multiple accesses to the prediction tables in each machine cycle. We found that most benchmarks have very low overflow rate in the queues of the value prediction table with a queue size of 32. The overall required hardware overhead of the proposed scheme is reasonably small. We found that the PVC prediction scheme is not very sensitive to the PVC size.

We also examine the effect of potential delay in updating the value prediction tables which is the main cause of stale prediction values and mispredictions. We found that, in a 8-issue processor, 36% of instructions access the same value prediction table entry again within 5 cycles, and 22% of instructions do that within 2 cycles. Unless the prediction tables can be quickly updated, especially for the Stride type and the Two-level type, those prediction tables could potentially contain stale prediction values. We examine the performance of some improvement schemes, such as attaching an age counter and using speculative updates for the Stride type and Two-level type. We found most of them not very effective due to the short average duration between two succesive accesses to the same table entry as mentioned above, and the relatively long latency required to dynamically classify access types. Future work should include using compiler analysis to identify those access types. It could potentially eliminate most of such latency in dynamic classification.

Acknowledgement

This work was supported in part by the National Science Foundation under grant nos. MIP-9610379 and EIA-9971666, and a gift from the Intel Corporation. Sang-Jeong Lee was supported in part by Soonchunhyang Univerity for his visit to the University of Minnesota.

References

[1] D.Burger and T.Austin, "The Simplescalar Tool Set, Version 2.0", *Technical Report CS-TR-97-1342*, University of Wisconsin, Madison, June 1997

[2] B.Calder, G.Reinman and D.Tullsen, "Selective Value Prediction", *Proceedings of the 26th International Symposium on Computer Architecture (ISCA-26)*, May 1999.

[3] F.Gabbay, and A.Mendelson, "Speculative Execution Based on Value Prediction", *EE Department TR 1080*, Technion– Israel Institute of Technology, Nov. 1996

[4] F.Gabbay, and A.Mendelson, "The Effect of Instruction Fetch Bandwidth on Value Prediction", *Proceedings of the 25th International Symposium on Computer Architecture (ISCA-25)*, p.272-281, 1998.

[5] J.Gonzalez and A.Gonzalez, "The Potential of Data Value Speculation to Boost ILP", *Proceedings of the International Conference on Supercomputing*, 1998.

[6] T.Heil, Z.Smith, and J.Smith, "Improving Branch Predictors by Correlating on Data Values", *Proceedings of the 32nd International Symposium Microarchitecture (MICRO-32)*, Nov. 1999.

[7] J.Huang and D.Lilja, "Exploiting Basic Block Value Locality with Basic Block Reuse", *Proceedings of the 5th International Symposium on*

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.

High Performance Computer Architecture (HPCA-5), 1999.

[8] M.Johnson, *Superscalar Microprocessor Design*, Prentice Hall, 1991

[9] S.Lee and P.Yew, "Decoupled Value Prediction on Trace Processors", *Proceedings of the 6th International Symposium on High Performance Computer Architecture (HPCA-6)*, 2000.

[10] M.Lipasti, and J.Shen, "Exceeding the Limit via Value Prediction", *Proceedings of the 29th International Symposium on Microarchitecture (MICRO-29)*, Dec. 1996.

[11] M.Lipasti, *Value Locality and Speculative Execution*, Ph.D. thesis, Carnegie Mellon University, April 1997.

[12] R.Rakvic, B.Black, and P.Shen, "Completion Time Multiple Branch Prediction for Enhancing Trace Cache Performance", Proceedings of the 27th Annual International Symposium on Computer Architecture (ISCA-27), June 2000.

[13] G. Reinman, T. Austin, and B. Calder, "A Scalable Front-End Architecture for Fast Instruction Delivery,"*Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA-26)*, May 1999.

[14] E. Rotenberg, S.B., and J.Smith, "Trace Cache: a Low Latency Approach to High Bandwidth Instruction Fetching", Technical Report: University of Wisconsin - Madison Technical Report #1310, Apr. 1996.

[15] B. Rychlik, J.Faistl, B.Krug, A.Kurland, J.Jung, Miroslav, N.Velev, and J.Shen, "Efficient and Accurate Value Prediction Using Dynamic Classification", *Technical Report of Microarchitecture Research Team in Dept. of Electrical and Computer Engineering*, Carnegie Mellon Univ., 1998.

[16] B.Rychlik, J.Faistl, B.Krug, and J.Shen, "Efficacy and Performance Impact of Value Prediction," *Parallel Architectures and Compilation Techniques*, Paris, Oct. 1998.

[17] T.Sato, "Analyzing Overhead of Reissued Instructions on Data Speculative Processors", *Workshop on Performance Analysis and its Impact on Design Held in Conjunction with ISCA-25*, 1998.

[18] Y. Sazeides, and J.Smith, "The Predictability of Data Values", *Proceedings of the 30th International Symposium on Microarchitecture (MICRO-30)*, Dec. 1997.

[19] K.Skadron, M.Martonosi, and D.Clark. "Speculative Updates of Local and Global Branch History: A Quantitative Analysis," Journal of Instruction-Level Parallelism, vol. 2, Jan. 2000.

[20] T.Sherwood and B.Calder, "Loop Termination Prediction", *Technical Report CS99-639 of University of California, San Diego, Dept. of Computer Science and Engineering*, Dec. 1999

[21] K.Wang, M.Franklin, "Highly Accurate data value Predictions using Hybrid Predictor," *Proceedings of the 30th International Symposium on Microarchitecture (MICRO-30)*, Dec. 1997.

[22] T. Yeh and Y.Patt, "Two-Level Adaptive Branch Prediction," *Proceedings of the 24th International Symposium Microarchitecture (MICRO-24)*, Nov. 1991.

Authorized licensed use limited to: N.C. State University Libraries - Acquisitions & Discovery S. Downloaded on April 24,2026 at 01:23:27 UTC from IEEE Xplore. Restrictions apply.